PayBench: A Pre-Registered Methodology and Calibrated Mock Harness for Agent-to-Agent Payment-Rail Settlement Finality

Michael Blake
Independent researcher | everydayai.link

Methodology v1.2 (frozen 2026-06-06). Preprint v1.

Abstract

This preprint describes a pre-registered measurement methodology and a calibrated *mock* harness; no real-rail funds are moved and no production rail-by-rail ranking is derived or published here. PayBench is an evaluation methodology and benchmark for agent-to-agent (A2A) payment rails. Its first dimension is *settlement-finality*: the wall-clock time from payment submission to the point at which an autonomous agent may rely on the payment for an irreversible downstream action. We make three contributions. First, a *reliance-level doctrine* for defining finality across heterogeneous rails (deterministic BFT close, probabilistic rooting, optimistic-rollup soft finality, hash-time-locked preimage release), pairing each rail's ecosystem-canonical reliance level with an explicit *trust/equivalence class* so that wall-clock comparisons never masquerade as security-model equivalence. Second, a comparative statistical method—Bradley–Terry maximum-likelihood estimation over pairwise finality races, complemented by pass@$k = P(\text{finality} \leq k)$ and Wilson lower-bound confidence intervals—with explicit caveats on its behaviour under cluster separation. Third, a cryptographic *pre-registration* protocol (OSF DOI + a transparency-log entry + an independent timestamp anchor + a signed source tag + this preprint) that fixes the design before any scored run, and that cleanly decouples *methodology* pre-registration from the later *real-rail data* pre-registration. The methodology and a reproducible mock harness are released as an open tool; publication of real-rail rankings is deferred.

1 Introduction

Agentic payment rails are proliferating—x402 on multiple chains, the Machine Payments Protocol (MPP) on different settlement rails, mandate/authorization layers such as AP2, and others. An autonomous agent transacting A2A must answer one question above all others before it acts on a payment: *when can I rely on this?* That is the settlement-finality question, and it gates every downstream action the agent takes. PayBench exists to define and publish the canonical, comparative metric for it.

The benchmark is deliberately *comparative*, not absolute: it ranks rails against each other on a dimension, with a stated confidence interval, under a protocol fixed in advance. The comparative framing is what makes a Bradley–Terry method the right tool, and what keeps the claims defensible.

Two design commitments shape everything below. (i) *Pre-registration.* The rail set, per-rail finality definitions, trial counts, metric definitions, the random seed, and the input fixture hashes are committed—cryptographically—before any scored run, so that no element can later be accused of having been chosen to flatter a rail. (ii) *A calibrated mock harness.* The released tool ships with

calibrated mock-data fixtures as its test harness, exercising the full measurement and statistics pipeline against realistic distributions *without* publishing a real-rail ranking. Running the method against production rails and publishing rail-by-rail scoring is deferred.

2 What PayBench measures

PayBench answers three different *kinds* of question and is explicit about which is which (a *hybrid-tiered* schema):

- **Tier 1 — benchmarked.** Dimensions PayBench runs trials on, pre-registers, and ranks with confidence intervals. Settlement-finality is the first; authorization latency and fee predictability are roadmap-committed.
- **Tier 2 — factual lookup.** Live, queryable capability/reference data per rail (fees, supported assets, custody model, the auth/dispute/refund/sanctions capability bundle), stated as fact, not scored.
- **Tier 3 — roadmap.** Dimensions named for transparency about direction, carrying no measurements.

The tiering is an honesty mechanism: measured, looked-up, and roadmapped claims stay visibly distinct, so nothing reads as a result that is not one.

3 Settlement-finality

Definition. Settlement-finality is the wall-clock time from payment *submission* to the point at which the value transfer is irreversible on the rail—the moment the payee can rely on the funds without risk of reversal.

The reliance-level doctrine. Rails do not share a single cryptographic finality model, so no single uniform threshold can be pinned across all of them. Instead PayBench pins each rail to its *ecosystem-canonical reliance level*—the point that rail's own protocol/documentation designates as the level at which a builder may rely on the payment for an irreversible downstream action. This is the correct doctrine for an *agent-DX* benchmark: it measures the moment an autonomous agent is told it can act.

The doctrine's sharp edge, named openly. Because the canonical reliance level differs by rail, the thresholds are not equi-conservative: an optimistic-rollup's canonical level is *optimistic* (soft finality), whereas a probabilistic L1's canonical level may be *conservative* (a rooted/finalized commitment rather than an optimistic one). A uniform-optimistic or a uniform-irreversibility doctrine would each be internally symmetric but would each misrepresent at least one rail's own builder guidance. PayBench takes the per-rail-canonical doctrine and *names* the asymmetry rather than hiding it. To make it impossible to miss, every rail carries a **trust/equivalence-class** label (Table 1)—a centralised-sequencer soft-commit is not the same security object as a decentralised-supermajority cryptographic root, even when both are that rail's canonical reliance point. The comparison is wall-clock-to-reliance, not security-model-to-security-model, and the method says so.[1]

[1] This doctrine was upheld 3–1 by an independent four-model adversarial review against the uniform-optimistic alternative, on the ground that ranking a rail at its *optimistic* commitment would publish a threshold that rail's own

Table 1: Trust/equivalence classes (illustrative of the doctrine; per-rail finality definitions and calibration parameters are in the repository, not reproduced here—see §5).

Finality mechanism	Trust / equivalence class
Optimistic-rollup soft finality (1 block)	Optimistic soft-commit (centralised sequencer promise)
Deterministic ledger/BFT close	Deterministic BFT
Probabilistic rooted commitment	Cryptographic root (decentralised supermajority)
Hash-time-locked preimage release	HTLC claim (possibly via hosted-operator ceremony)

Unit. Seconds, wall-clock, from request to the *agent-perceived finality signal*—the point at which the agent is told it may act, which per rail adapter may be an HTTP response, a webhook, or a polled state transition (not necessarily the first acceptance response). This deliberately bundles the rail's protocol/transport envelope with on-chain settlement, because that full round-trip is what the agent actually experiences and waits on.

Coverage note. Authorization/mandate layers that do not settle independently (their finality reduces to an underlying rail) are excluded from the finality dimension and measured on an authorization-latency dimension instead, where they are well-defined. Racing such a layer against its own underlying rail would violate the independence the statistical model assumes.

4 Statistical method

Bradley–Terry MLE. For each unordered rail pair we run repeated finality *races*: in a trial, both rails execute the same scenario against the same fixture and the rail reaching finality first wins. Aggregating wins/losses across all trials and pairs, Bradley–Terry MLE produces a latent strength per rail,

$$P(i \succ j) = \frac{e^{\beta_i}}{e^{\beta_i} + e^{\beta_j}},$$

a single comparative ranking internally consistent across all pairwise comparisons.

Caveat—separation. When rails fall into widely separated latency regimes, slower rails essentially never beat faster ones and the MLE approaches complete separation; a symmetric smoothing prior keeps it finite. In that regime the BT strength *magnitudes* are an artefact of the prior and the trial count—only the *rank order* and any *within-regime* differences are interpretable. We therefore do not present fine-grained strengths as precise, and publish the raw pairwise win/loss matrix so a reader can verify the ranking is data-driven, not prior-driven. pass@k is the metric that actually discriminates in this regime.

pass@k**.** Operationally an agent needs an absolute answer: "will this rail settle within my SLA?" We report pass@$k = P(\text{finality} \leq k \text{ seconds})$ for a frozen grid $k \in \{2, 3, 5, 10, 15, 20\}$ s. The grid is justified on *external agent-SLA grounds*—2 s as an interactive-HTTP responsiveness threshold, 3/5 s as synchronous-call budgets, 10 s as a gateway/timeout boundary, 15/20 s as batch/background-settlement budgets—not reverse-engineered from the data. It spans the full observed finality range so it discriminates both within the fast-settling and within the slow-settling regimes, with 10 s as the separator. The grid is frozen with the rest of the design and is not re-fit per run.

guidance disavows for irreversible reliance—i.e. measurement invalidity.

Honesty note on grid timing. Calibration data was collected *before* the methodology freeze, so the grid was finalised with the calibrated distributions in hand; the *mock* run therefore validates the measurement/statistics *pipeline* against known inputs rather than testing an independent hypothesis. The grid stands on the external SLA rationale above, and the *same frozen grid carries unchanged to the eventual real-rail run*, where it is fixed before that data exists. Pre-registration here prevents p-hacking on the real-rail run; it does not, and is not claimed to, retroactively blind the designers to the calibration data.

Wilson lower bounds, and their scope. All proportions (pairwise win rates and pass@k) are reported with Wilson score intervals, leading with the 95% lower bound as the conservative figure; the Wilson interval is well-behaved near 0 and 1, where a normal approximation misleads. Two scope caveats: (i) the intervals are *pointwise* 95%, with no family-wise or false-discovery correction across the set, as they are reported as independent descriptive metrics rather than a joint hypothesis test; (ii) on *mock* fixtures a Wilson interval captures the Monte-Carlo sampling error of the harness, not real-world network variance—which is a separately disclosed limitation.

5 Experimental design and the calibrated mock harness

The settlement-finality benchmark races the rails that settle independently. For the present configuration this is $C(5, 2) = 10$ unordered pairs at 500 trials per pair = 5,000 symmetric trials. A single published master seed drives all stochastic fixture selection and trial ordering (stdlib Mersenne Twister; no wall-clock, no OS entropy), and every fixture is content-addressed (sha256) so a trial records, and the harness re-verifies, the exact fixture bytes it consumed. The harness is pure standard library, removing cross-version RNG drift.

Calibrated mock fixtures. Each fixture is a seeded log-normal population whose location and scale parameters are derived from real reference data—rail documentation, public telemetry, and first-party testnet/devnet/regtest observations. The harness is therefore *meaningful*: it exercises the full pipeline against realistic, first-party-anchored distributions.

Perimeter discipline—why no per-rail numbers appear here. A seconds-valued, fastest-to-slowest table of real first-party measurements *is* a rail ranking, which the publication posture defers. Accordingly this preprint reports *no* per-rail finality medians and *no* ranking. The calibration parameters (log-space μ, σ per rail), full per-source provenance, the harness, and the deterministic run report are released in the repository; this paper documents the *method*. The released mock run demonstrates the pipeline; it is not, and is not presented as, a real-rail result.

Disclosed limitations (no silent caps). The following are committed *into* the frozen method so a later real-rail run is measured against a prior that already names them: (1) quiet testnet/devnet/regtest conditions understate mainnet congestion tails, so the fixture scale parameters are an optimistic tail estimate; (2) one rail's fixture is a manual-path *lower bound* on its spec path; (3) a testnet-calibrated rail may differ from mainnet in empirical latency despite a shared consensus algorithm. Central tendencies are high-confidence first-party; tails are the disclosed soft spot. For the real-rail run, parameter uncertainty in the scale estimate (a modest per-rail sample) should additionally be bootstrapped and propagated, and deterministic-finality rails should use a discrete (constant-plus-jitter) distribution rather than log-normal.

6 Pre-registration protocol

Pre-registration is the load-bearing artefact: it is what defends a comparative leaderboard against selective-trial and threshold-shopping critique. We pre-register, before any scored run, the rail set, per-rail finality definitions, trial counts, metric definitions (BT + pass@k grid + Wilson), the RNG seed, and the input fixture hashes. The concrete frozen values are committed as a content manifest; the manifest's own hash is the single value the anchors below commit to.

Cryptographic grounding (defence-in-depth). The manifest hash is committed via three independent trust anchors plus two corroborating records: an OSF pre-registration resolving to a DOI; a signature over the manifest logged to a public transparency log; an independent timestamp anchored to a public blockchain; a signed source-control tag on the frozen commit (hardware-key signed); and this preprint. The triad (DOI + blockchain timestamp + transparency-log entry) provides no single point of trust; the transparency-log entry and timestamp proof are independently sufficient to establish the freeze date even if the DOI host is unavailable. Bulk trial telemetry, when real-rail runs occur, is signed with a Merkle root plus the same signing mechanism.

Methodology vs. data pre-registration. This artefact pre-registers the *measurement method and the calibrated mock baseline*. The eventual real-rail run is a *separate* pre-registration: its specific production-fixture hashes and run configuration are committed, with the same anchor stack, *before* that run executes. Decoupling the two closes the "ran the real benchmark many times and registered only the best" attack—the real-rail design is frozen ahead of the real-rail data, exactly as this method was frozen ahead of any real-rail data.

Verification. The released manifest verifies every input file by sha256; the harness re-derives the mock-pipeline verification hash byte-for-byte from the frozen fixtures (no wall-clock is written into the report). **This verification hash certifies pipeline reproduction; it is not a real-rail ranking result.** Only version 1 of this preprint is the canonical frozen artefact; later revisions, if any, are post-freeze errata.

7 Related work

Adjacent efforts touch this ground—registries of payment-capable services, multi-rail routing and analytics layers, and service-discovery surfaces. PayBench's distinct contribution is *methodological*: a pre-registered, comparative, settlement-finality benchmark published from a non-custodial, rail-neutral posture, with the design fixed cryptographically before any scored run. A neutral, sourced, dated related-work treatment naming specific efforts belongs in the camera-ready version; it is omitted from this preprint to keep the instrument neutral.

8 Conclusion

PayBench operationalises the dominant rail-DX question for agent-to-agent commerce—when can the agent rely on a payment?—as a comparative, pre-registered benchmark, with a finality doctrine that is honest about cross-rail trust asymmetry and a statistical method that is honest about its own degeneracies. The methodology and a reproducible calibrated-mock harness are released now; real-rail rankings follow, under a separate data pre-registration, when the publication posture permits.

Availability. Methodology of record, harness, calibration provenance, and the pre-registration manifest are in the project repository at the frozen commit referenced by the signed tag `paybench-prereg-v1.2`. The manifest hash committed by the anchors is `sha256:a5f6feb4...d3a6f` (full value in the repository manifest).